Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Development of a Modular 3D Game Engine Framework: Core Architecture, Editor Tooling and Scene Management in C++

Brian Walsh B.Sc. Software Development, ATU Galway

GitHub: https://github.com/ATU-Game-Engine/GameEngine

Introduction

This project presents the development of a modular 3D game engine framework built in C++ and OpenGL as part of a collaborative Final Year Project. Three developers each took responsibility for a distinct component: framework, rendering and physics.

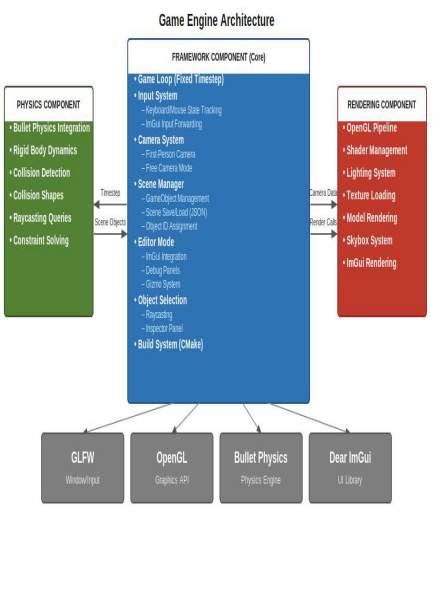
The framework is the foundational integration layer, coordinating the rendering and physics modules. It provides timing infrastructure, input management, camera control, editor tooling and scene management. The rendering pipeline was developed independently by Radoslaw Rodak and the physics system by Liam Ryan.

Building from scratch reveals architectural decisions and real-time constraints that commercial engines such as Unity and Unreal Engine abstract away. This provided valuable insight into 3D systems programming.

System Overview

The engine follows a modular design with the framework at the centre, coordinating rendering and physics through well-defined interfaces. The framework initialises the OpenGL context via GLFW, drives the fixed-timestep physics loop and exposes a scene graph of GameObjects to the renderer.

Transform synchronisation occurs through the shared scene system: during gameplay, physics controls object positions; in editor mode, authored transforms take precedence. The framework passes timestep data to physics and camera matrices to the renderer.



Tech Stack

Language: C++
Graphics API: OpenGL
Windowing / Input: GLFW
Physics: Bullet Physics
UI: Dear ImGui
Build System: CMake / vcpkg
Version Control: GitHub
IDE: Visual Studio 2022

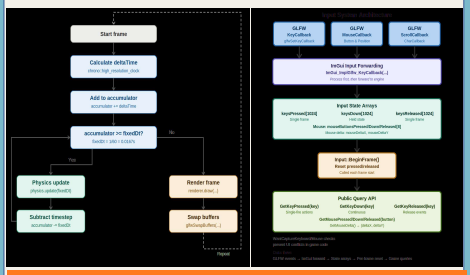
External libraries limited to windowing, physics, UI and build tooling. The core game loop, input system, scene manager and editor are entirely custom C++.

References

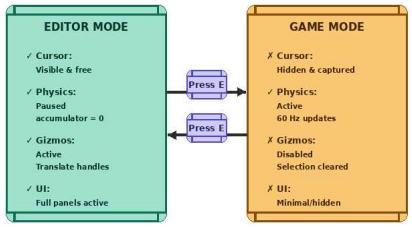
[1] GLFW Documentation. glfw.org, 2024
[2] O. Cornut. Dear ImGui. github.com/ocornut/imgui, 2024
[3] Unity Technologies. GameObject. docs.unity3d.com, 2024
[4] Unity Technologies. Physics.Raycast. docs.unity3d.com, 2024

Framework Architecture

The framework encompasses five subsystems: a fixed-timestep game loop, a three-state input system, a camera system, a scene manager with JSON serialisation and an editor mode with Dear ImGui integration. Transform gizmos and mouse-based raycasting for object selection.



Key Features



Fixed-Timestep Game Loop
An accumulator pattern maintains consistent 60 Hz physics updates independent of rendering frame rate. A 250ms frame-time cap prevents the spiral-of-death during performance hitches. In editor mode, the accumulator is discarded each frame to freeze physics while scene content is authored.

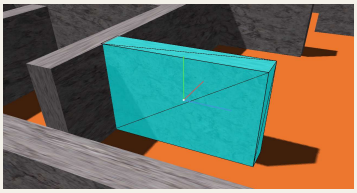
Input System
Three static arrays (keysPressed[1024], keysDown[1024], keysReleased[1024]) track pressed, held and released states per frame. GLFW callbacks populate the arrays; BeginFrame() clears transient states at the start of each frame. All events forward to Dear ImGui before engine processing with WantCaptureKeyboard/WantCaptureMouse checks preventing UI click-through.

Editor Mode
The E key toggles between Editor and Game modes, reconfiguring cursor visibility, physics activation, input routing and UI visibility. Editor mode pauses physics, shows transform gizmos and enables ImGui panels. Game mode hides the cursor, captures mouse input and activates all physics bodies.

Object Selection and Raycasting
Mouse clicks transform from screen space through NDC, clip, eye and world space via inverse projection and view matrices. Ray-AABB intersection uses the slab method, tracking the closest hit to select the frontmost object. The gizmo system enables axis-constrained translation by projecting mouse movement onto a drag plane perpendicular to the active axis.

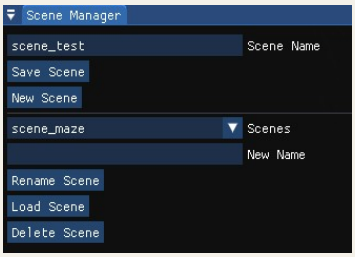
Scene Serialisation
Scenes persist as human-readable JSON with 4-space indentation. Each GameObject serialises its ID, name, tags, shape type, transform (position, rotation as quaternion, scale), rendering configuration and physics parameters. The Scene Manager UI restricts save, load, rename and delete to editor mode with confirmation popups on deletion and F5/F9 keyboard shortcuts for rapid iteration.

Results

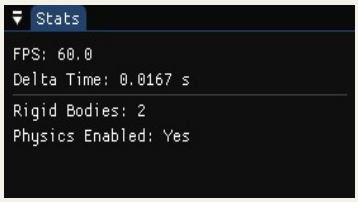


Interactive transform gizmo showing red (X), green (Y) and blue (Z) axes for axis-constrained object manipulation in the 3D scene

Inspector panel showing selected object (ID: 2, Name: Player) with live-editable transform data, physics configuration and tag system



Scene Manager UI providing save, load, rename and delete operations with dropdown list of available scenes



Stats panel confirming fixed-timestep performance: FPS 60.0, Delta Time 0.0167s with rigid body and physics state tracking

Conclusion

All five objectives were successfully delivered. The fixed-timestep accumulator maintains deterministic 60 Hz physics independent of rendering frame rate, confirmed by runtime measurements of 16.67ms delta time and exactly 60 physics steps per second. The editor mode provides comprehensive Dear ImGui debug tools with interactive transform gizmos.

The three-state input system isolates single-frame events from held states and prevents UI click-through. The raycasting system accurately transforms screen-space clicks into world-space selections via AABB slab intersection. JSON scene serialisation persists complex object hierarchies with full round-trip fidelity.

Key insight: dual-mode state management proved complex, requiring orchestration of cursor, physics, input and selection state. Future work includes spatial partitioning, rotation/scale gizmos and undo/redo infrastructure.

Acknowledgements

Supervised by Dr. Aaron Hurley, Atlantic Technological University Galway.